```
#%%
# door_open() called, clear int and ext reqs for current floor
def door_open(pos, dir, ext_req, int_req):
  # if ext_req[pos]==True:
  #   ext_req[pos] = False

  ext_req[pos] = False

  if pos in int_req:
    int_req.remove(pos)

  return ext_req, int_req

door_open(1, 1, [False, True, True, True, False], [1, 2])
# expected answer: ([False, False, True, True, False], [2])

def closest_floor(pos,dir,requests):
  # convert req floors into differences

  diff_list = []
  for j in range(len(requests)):
    diff = requests[j]-pos
    diff_list.append(diff)

  if dir==1:
    return min([x for x in diff_list if x>0])+pos
  elif dir==-1:
    return max([x for x in diff_list if x<0])+pos
  else:
    return (0,pos)


def move(pos,dir,ext_req,int_req):

  combined_req = []
  for i in range(len(ext_req)):
    if ext_req[i]==True:
      combined_req.append(i)

  combined_req.extend(int_req)
  clean_combined_req = sorted(list(set(combined_req)))

  if len(clean_combined_req)==0: #no requests
    return (0,pos)
  else:
    return (dir, closest_floor(pos,dir,clean_combined_req))

move(2, 1, [False, False, False, True, False], [0, 1])
#expected output: (1, 3)

#%% Qn 2

'''You have just received your Semester 2 results. Write a function that updates your cumulative CAP after adding Semester 2 grades to your existing CAP.
Inputs:
prev_cap (float): this is your CAP before semester 2
prev_mc (int): total MCs completed before semester 2
sem_results: a list whose first element is the student ID (string), followed by (grade_str, mc) tuples. Example:  ["ID1234A", ("A", 4), ("B", 4)]
Use the grading policy provided at NUS Grading Policy to calculate the CAP for each grade.
Output:
Return a tuple (new_cap, total_mc) after the update, where:
new_cap is the updated cumulative CAP, rounded to one decimal place, and total_mc is the new total number of MCs completed.
'''

def updated_cap(prev_cap, prev_mc, sem_results):
  pass


# updated_cap(3.8, 20, ["ID1234A", ("A", 4), ("B", 4)])
# should return (3.9, 28)
```

```
#%% Qn 3 - Queens on chessboard

'''Inputs: A list of (row, column) positions for each of the queens.
Return: Empty tuple () if no pair of queens attack each other. Otherwise, return a tuple (i, j) indicating the indices of any attacking pair.
Example:
queens_attack([(2, 5), (4, 2), (0, 6)]) should return (1, 2)'''

def queens_attack(queens):
  # q_pair = []
  for i in range(len(queens)):
    for j in range(i,len(queens)):
      if i==j:
        continue
      if los(queens[i], queens[j])==True:
        # q_pair.append((i,j))
        return (i,j)
  return ()


def los(queen1, queen2):
  dy = queen1[1] - queen2[1]
  dx = queen1[0] - queen2[0]
  if dy==0 or dx==0:
    return True
  elif dy/dx==1 or dy/dx==-1:
    return True
  else:
    return False

# queens = [(2, 5), (4, 2), (0, 6)]
# print(los(queens[0], queens[2]))
queens_attack([(2, 5), (4, 2), (0, 0)])
#%% Qn4

'''Input: A list of numeric values, and the threshold
Output: Return a list of indices of elements that are significant local peaks. Return an empty list if there is no significant local peaks.
Example:
significant_peaks([1.0, 1.1, 2.0, 1.2, 1.5, 2.1, 1.6, 1.0], 0.5) should return [2]
'''
def significant_peaks(x, threshold):
  local_peaks=[]
  for i in range(1,len(x)-1):
    try:
      diff1 = (x[i]-x[i-1])/x[i-1]
      diff2 = (x[i]-x[i+1])/x[i+1]

      if diff1>=threshold and diff2>=threshold:
        print(diff1, diff2)
        local_peaks.append(i)
    except:
      continue #zero error
  return local_peaks

significant_peaks([1.0, 10.1, 2.0, 10.2, 1.5, 2.1, 1.6, 1.0], 0.5)
#%% Qn5
''' Your manager is preparing a visual alignment test for a new printer model (the printer supports both text and simple patterns, though only text mode is used here).
Write a function print_pattern(n, c1, c2) that generates a pattern with n rows to be sent to the printer for inspection.
The i-th row should contain i occurrences of c1, followed by (n – i) occurrences of c2.
Each row should be separated by a newline character ('\n').
If n is 0, simply return an empty string.
The function should return the pattern as a single string (do not print it directly).
The pattern does not need to be centered or padded; the printer alignment will handle that automatically.
'''

def print_pattern(n,c1,c2):
  i=1
  final = ''

  while i <= n:
    shengzi = str(c1*i) + str(c2*(n-i))
    if i <= (n-1):
```

```
    shengzi += '\n'
  final += shengzi
  i += 1

 return final

print_pattern(3, '*', '-')
# #expected output: '*--\n**-\n***'

#%% Qn6
enrollments = {
 'TCX1001': {'Alice', 'Bob', 'Charlie'},
 'TCX1002': {'Alice', 'Bob', 'Charlie'},
 'TCX1003': {'Bob', 'Charlie', 'Diana', 'Ethan'},
 'TCX1004': {'Alice', 'Charlie'}
}

'''Write a Python function to find all pairs of students who share at least 3 common courses. Below are the example input and the expected result.
func(enrollments) should return
{
  frozenset({'Alice', 'Charlie'}): {'TCX1001', 'TCX1002', 'TCX1004'},
  frozenset({'Bob', 'Charlie'}): {'TCX1001', 'TCX1002', 'TCX1003'}
}
'''
def func(enrollments):

 enrollments_inverse = {}

 #dict with students as keys
 for course, students in enrollments.items():
  for student in students:
   if student not in enrollments_inverse.keys():
    enrollments_inverse[student] = set()
   enrollments_inverse[student].add(course)

 # comparing nested loops
 names = sorted(list(enrollments_inverse.keys()))
 result = {}

 for i in range(len(names)):
  for j in range(i+1,len(names)):
   s1, s2 = names[i], names[j]
   # print(s1, s2)
   common_course = enrollments_inverse[s1] & enrollments_inverse[s2]
   # print(f"common_course: {len(common_course)}")
   if len(common_course)>=3:
    result[frozenset({s1,s2})] = common_course

 return(result)

func(enrollments)


#%% Qn7 - NEA readings

'''Write a Python function that retrieves temperature readings from Singapore National Environment Agency (NEA)'s data. The input is the name (or part of the name) of a weather station or location. It should match the input against the available station names (case-insensitive), and return a list of tuple consists of the station name and the temperature. If no station name matches the input, return an empty list. Example of NEA data is already given.'''

data = {
 "stations": [
  {
   "id": "S109",
   "deviceId": "S109",
   "name": "Ang Mo Kio Avenue 5",
   "location": {
    "latitude": 1.3764,
    "longitude": 103.8492
   }
  },
  {
   "id": "S106",
```

```
  "deviceId": "S106",
  "name": "Pulau Ubin",
  "location": {
    "latitude": 1.4168,
    "longitude": 103.9673
  }
},
{
  "id": "S117",
  "deviceId": "S117",
  "name": "Banyan Road",
  "location": {
    "latitude": 1.256,
    "longitude": 103.679
  }
},
{
  "id": "S107",
  "deviceId": "S107",
  "name": "East Coast Parkway",
  "location": {
    "latitude": 1.3135,
    "longitude": 103.9625
  }
},
{
  "id": "S115",
  "deviceId": "S115",
  "name": "Tuas South Avenue 3",
  "location": {
    "latitude": 1.29377,
    "longitude": 103.61843
  }
},
{
  "id": "S102",
  "deviceId": "S102",
  "name": "Semakau Landfill",
  "location": {
    "latitude": 1.189,
    "longitude": 103.768
  }
},
{
  "id": "S60",
  "deviceId": "S60",
  "name": "Sentosa",
  "location": {
    "latitude": 1.25,
    "longitude": 103.8279
  }
},
{
  "id": "S50",
  "deviceId": "S50",
  "name": "Clementi Road",
  "location": {
    "latitude": 1.3337,
    "longitude": 103.7768
  }
},
{
  "id": "S44",
  "deviceId": "S44",
  "name": "Nanyang Avenue",
  "location": {
    "latitude": 1.34583,
    "longitude": 103.68166
  }
},
{
  "id": "S43",
```

```
    "deviceId": "S43",
    "name": "Kim Chuan Road",
    "location": {
      "latitude": 1.3399,
      "longitude": 103.8878
    }
  },
  {
    "id": "S24",
    "deviceId": "S24",
    "name": "Upper Changi Road North",
    "location": {
      "latitude": 1.3678,
      "longitude": 103.9826
    }
  },
  {
    "id": "S06",
    "deviceId": "S06",
    "name": "Paya Lebar",
    "location": {
      "latitude": 1.3524,
      "longitude": 103.9007
    }
  },
  {
    "id": "S111",
    "deviceId": "S111",
    "name": "Scotts Road",
    "location": {
      "latitude": 1.31055,
      "longitude": 103.8365
    }
  }
],
"readings": [
  {
    "timestamp": "2025-11-16T17:53:00+08:00",
    "data": [
      {
        "stationId": "S109",
        "value": 24.9
      },
      {
        "stationId": "S106",
        "value": 24
      },
      {
        "stationId": "S117",
        "value": 25.1
      },
      {
        "stationId": "S107",
        "value": 24.9
      },
      {
        "stationId": "S115",
        "value": 26.5
      },
      {
        "stationId": "S102",
        "value": 24.6
      },
      {
        "stationId": "S60",
        "value": 24.1
      },
      {
        "stationId": "S50",
        "value": 23.3
      },
      {
```

```
        "stationId": "S44",
        "value": 25.4
      },
      {
        "stationId": "S43",
        "value": 23.9
      },
      {
        "stationId": "S24",
        "value": 24.4
      },
      {
        "stationId": "S06",
        "value": 23.7
      },
      {
        "stationId": "S111",
        "value": 24.3
      }
    ]
  }
],
"readingType": "DBT 1M F",
"readingUnit": "deg C"
}
```


```
def func(location,data):
  result = {}
  lst = []

  for station in data["stations"]:
    station_name = station["name"]
    if location.lower() in station_name.lower():
      station_id = station["id"]
      temp_value = temp(station_id)
      if station["name"] not in result.keys():
        pair = (station_name, temp_value)
        lst.append(pair)

  return sorted(lst)


def temp(station_id):
  for reading in data["readings"]: # extract temp from reading dict
    for i in range(len(reading["data"])):
      if reading["data"][i]['stationId'] == station_id:
        temp = reading["data"][i]['value']
        return temp

set(func('s',data))
# set(func('s', data))
#{('East Coast Parkway', 24.9), ('Semakau Landfill', 24.6), ('Scotts Road', 24.3), ('Sentosa', 24.1), ('Tuas South Avenue 3', 26.5)}


#%% Qn8

parsed_elements = [('0', 'html', '', -1, -1),
('1', 'head', '', 0, -1),
('2', 'title', 'Sample Page', 1, -1),
('3', 'body', '', 0, 1),
('title', 'h1', 'My Sample Page', 3, -1),
('main', 'div', '', 3, 4),
('4', 'p', 'This is the first paragraph in main.', 5, -1),
('5', 'div', '', 5, 6),
('nested', 'p', 'Here is a nested paragraph inside inner div.', 7, -1),
('6', 'p', 'This is the 2nd paragraph in main.', 5, 7)]

def insert(id, tag, text, parent_idx, prev_idx):
  entry = (id, tag, text, parent_idx, prev_idx)
  parsed_elements.append(entry)
  return "entry inserted"
```

```
insert('test', 'p', 'between 1st and 2nd', 5, 7)
print(parsed_elements)

def delete(id): # set the parent_index of the affected element to -1.
  pass
```

expected [('0', 'html', '', -1, -1), ('1', 'head', '', 0, -1), ('2', 'title', 'Sample Page', 1, -1), ('3', 'body', '', 0, 1), ('title', 'h1', 'My Sample Page', 3, -1), ('main', 'div', '', 3, 4), ('4', 'p', 'This is the first paragraph in main.', 5, -1), ('5', 'div', '', 5, 6), ('nested', 'p', 'Here is a nested paragraph inside inner div.', 7, -1), ('6', 'p', 'This is the 2nd paragraph in main.', 5, 10), ('test', 'p', ' between 1st and 2nd', 5, 7)]